

第11章 反射与动态特性

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解"运行时类型信息":为什么 C 没有反射,JS 为什么天然就有
- 掌握 typeof 的全部结果与三个著名坑(null、数组、函数)
- 掌握 instanceof 的语义与跨 realm 陷阱
- 掌握 Object 反射全家:keys/values/entries/assign/getOwnPropertyDescriptors
- 掌握属性描述符:writable/enumerable/configurable 与 defineProperty
- 掌握 Reflect 全套与动态属性访问(方括号、in、delete)
- 掌握生产判断:何时该用反射,何时不该,替代手段清单

💡 从 C 到 JavaScript:本章主题如何迁移

C 没有反射,运行时"不知道"自己的类型: `sizeof(x)` 是编译期常量,结构体字段名在编译后只是偏移量,不可能"遍历一个结构体的所有字段"。想模拟(如序列化),只能手写字段清单或生成代码。高级语言把这份能力带到运行时: **运行时类型信息(RTTI)**——Java 的反射、Python 的内省、JS 的 Object/Reflect 全家,都能在程序运行时"查看自己的结构"。

JS 的反射尤其彻底,因为对象就是"动态属性表": `Object.keys(obj)` 枚举属性、`obj[key]` 用变量取属性、`'k' in obj` 检查存在——这些在 C 里统统做不到(或要靠宏 hack)。配合属性描述符,你可以精确控制每个字段的读写权与可见性,这比 C 的 struct 成员权限丰富得多。

但反射是双刃剑:它把"编译期该查出的错"推迟到运行期,性能也打折扣(动态属性访问比静态慢一个数量级)。生产判断标准非常清晰: **框架层(序列化、ORM、DI、调试工具)用反射,业务热路径不用**。第11章的实验(手写 JSON 序列化器)正是反射的经典教学用例——你会在 60 行代码里体会到 C 程序员写序列化时"没有反射"的痛苦,以及 JS 的轻松。

📦 前置知识自查

- C 的 sizeof、结构体成员访问、offsetof(可选)
- 第1章 typeof 与八种类型;第6章原型链与 hasOwnProperty
- 第10章 Proxy/Reflect(本章是它的"无拦截版"补充)

🚀 本章学习路线

1. **第一阶段(类型探测)**:读 §11.1–§11.2,把 typeof/instanceof 的坑全部过一遍
2. **第二阶段(Object 反射)**:读 §11.3–§11.4,练习枚举与描述符操作
3. **第三阶段(Reflect 与生产)**:读 §11.5–§11.7,理解反射的代价与边界
4. **第四阶段(实验)**:完成章末"属性遍历打印器 + 手写 JSON 序列化器"
5. **验收标准**:能写出一个通用"对象 → JSON"的递归序列化器,并处理数组/嵌套/循环引用

本章知识导图

- 第11章 反射与动态特性
 - §11.1 typeof:八种结果与三个坑
 - §11.2 instanceof:原型链语义与跨 realm
 - §11.3 Object 反射全家:keys/entries/assign
 - §11.4 属性描述符与 defineProperty
 - §11.5 Reflect 全套与动态属性
 - §11.6 动态特性:方括号、in、delete、原型操纵
 - §11.7 生产判断:何时用反射,替代手段

§11.1 typeof:八种结果与三个坑

```
// 完整结果表(第1章已见,这里做"反射"视角的复习)
console.log(typeof 42);           // 'number'
console.log(typeof 42n);          // 'bigint'
console.log(typeof 's');          // 'string'
console.log(typeof true);         // 'boolean'
console.log(typeof undefined);    // 'undefined'
console.log(typeof Symbol());     // 'symbol'
console.log(typeof {});           // 'object'
console.log(typeof function(){}); // 'function'

// 坑一:null
console.log(typeof null);         // 'object' ← 语言 bug,判断 null 用 === null

// 坑二:数组
console.log(typeof []);           // 'object' ← 不能靠 typeof 区分数组
console.log(Array.isArray([]));   // true ← 正确姿势

// 坑三:函数也是对象,但 typeof 特殊
console.log(typeof async () => {}); // 'function'(async 函数也是 function)

// 生产模式:类型判定工具函数
function typeOf(x) {
  if (x === null) return 'null';
  if (Array.isArray(x)) return 'array';
  const t = typeof x;
  if (t === 'object') return 'object';
  return t;
}
console.log(typeOf(null), typeOf([]), typeOf({}), typeOf(42));
// null array object number
```

⚠ 常见误区 11.1:typeof 判"对象是否为空"

`typeof obj === 'object'` 不能判断"obj 有没有内容"——空对象 `{}` 也是 `'object'`。判断"空对象"要 `Object.keys(obj).length === 0`;判断"对象存在"用 `obj !== null`。`typeof` 只回答"类别",不回答"内容",这是反射类操作的第一课。

§11.2 instanceof:沿原型链的类型检查

```
// 语义:obj instanceof Ctor = Ctor.prototype 是否在 obj 的原型链上
class Animal {}
class Dog extends Animal {}
const d = new Dog();
console.log(d instanceof Dog);      // true
console.log(d instanceof Animal);    // true ← 沿链向上匹配
console.log(d instanceof Object);    // true ← 所有对象都匹配 Object
console.log({} instanceof Object);   // true
console.log({} instanceof Animal);   // false

// 基本类型的坑:number 原始值不是对象
console.log(42 instanceof Number);   // false ← 原始值没有原型链
console.log(new Number(42) instanceof Number); // true ← 包装对象才行
// 生产:基本类型判断用 typeof 或 Object.prototype.toString

// 跨 realm 陷阱(生产必知):
// iframe / worker / vm 里创建的数组,其 Array.prototype 与主 realm 不同
// [] instanceof 主realm的Array 可能为 false
console.log(Array.isArray([]));       // true ← 跨 realm 也安全
// 同理:instanceof 在"包装/代理/跨上下文"场景都不可靠,优先用鸭子检查或
// Object.prototype.toString.call(x) === '[object Array]'

// Object.prototype.toString:万能类型标签
console.log(Object.prototype.toString.call([])); // [object Array]
console.log(Object.prototype.toString.call(null)); // [object Null]
console.log(Object.prototype.toString.call(new Date())); // [object Date]
console.log(Object.prototype.toString.call(/re/)); // [object RegExp]
```

§11.3 Object 反射全家:枚举与复制

```
const user = { id: 1, name: '李雷', score: 92 };

// 三个枚举出口(注意只枚举"自身可枚举"属性)
console.log(Object.keys(user));      // [ 'id', 'name', 'score' ]
console.log(Object.values(user));    // [ 1, '李雷', 92 ]
console.log(Object.entries(user));   // [ ['id',1], ['name','李雷'], ['score',92] ]

// entries 配 for...of 是最常用遍历(ES2017+):
for (const [k, v] of Object.entries(user)) {
  console.log(`${k} = ${v}`);
}

// 从 entries 重建对象:
const copy = Object.fromEntries(Object.entries(user));
console.log(copy);                  // { id: 1, name: '李雷', score: 92 }

// assign:合并/拷贝(浅拷贝!)
const a = { x: 1 };
const b = { y: 2 };
const merged = Object.assign({}, a, b);
console.log(merged);                // { x: 1, y: 2 }
// 现代等价写法:{ ...a, ...b }(第2章展开运算符)

// 所有属性(含不可枚举、Symbol):getOwnPropertyNames / getOwnPropertySymbols
const special = {};
Object.defineProperty(special, 'hidden', { value: 1, enumerable: false });
console.log(Object.keys(special));   // [] ← 不可枚举不出现在 keys
console.log(Object.getOwnPropertyNames(special)); // [ 'hidden' ] ← 全量

// 所有属性的完整描述符:
console.log(Object.getOwnPropertyDescriptors(user));
// { id: { value: 1, writable: true, enumerable: true, configurable: true }, ... }
```

⚠ 常见误区 11.2:keys 不包含继承属性,也不包含 Symbol

`Object.keys` 只返回 自身可枚举的字符串键:继承的属性(原型上的)看不到(用 `for...in` 才看得到);`Symbol` 键看不到(用 `Reflect.ownKeys` 或 `getOwnPropertySymbols`);不可枚举的看不到(用 `getOwnPropertyNames`)。"我想拿到这个对象所有字段"要组合: `Reflect.ownKeys(obj)` 最全。

§11.4 属性描述符:三个开关

每个属性背后有三个开关:**writable**(可改值)、**enumerable**(出现在 `keys`/`for...in`)、**configurable**(可删除/重定义)。C 的 `struct` 成员没有这些概念,这是 JS 属性系统独有的一层"反射能力"。

```

const config = {};
Object.defineProperty(config, 'apiKey', {
  value: 'sk-abc123',
  writable: false,      // 只读
  enumerable: false,    // 不可枚举(日志/序列化时不会泄露)
  configurable: false,  // 不可删除、不可改描述符
});

// 读属性值(反射):
console.log(config.apiKey);      // sk-abc123
console.log(Object.keys(config)); // [] ← 不可枚举,序列化不泄露

// 写与删被拒绝:
'use strict';
try {
  config.apiKey = 'x';          // 严格模式抛 TypeError
} catch (e) {
  console.log('只读拦截:', e.name);
}
console.log(delete config.apiKey); // false ← configurable:false,删除无效

// 查询描述符:
console.log(Object.getOwnPropertyDescriptor(config, 'apiKey'));
// { value: 'sk-abc123', writable: false, enumerable: false, configurable: false }

// 生产用途:内部状态对外只读、敏感字段防泄露、框架字段防篡改
function createInternal(secret) {
  const state = { secret };
  Object.defineProperty(state, 'secret', { enumerable: false });
  return state;
}
const s = createInternal('机密');
console.log(JSON.stringify(s)); // {} ← secret 不会进 JSON!

```

工程建议 11.1:defineProperty 的三个生产场景

① 敏感字段(`enumerable: false`),防止 `JSON.stringify`/日志把密钥、令牌带出去——比"删除字段"更安全(代码里还能用);② 只读配置(`writable: false`),防止运行时被意外改写;③ 常量属性绑定。注意:`Object.freeze` 是"全部属性一键只读+不可配置"的快捷方式,区别是 `freeze` 不改 `enumerable`。

§11.5 Reflect 全套与动态属性

```
// Reflect 与 Object 反射的分工:Reflect 更"函数式"、返回值友好
const obj = { a: 1 };

console.log(Reflect.get(obj, 'a'));           // 1,替代 obj['a']
console.log(Reflect.has(obj, 'a'));           // true,替代 'a' in obj
console.log(Reflect.ownKeys(obj));             // [ 'a' ],最全的键(含 Symbol)
console.log(Reflect.deleteProperty(obj, 'a')); // true,返回布尔而非 undefined
console.log(obj);                             // {}

// 动态属性访问:方括号是反射的日常形态
const key = 'name';
const person = { name: '韩梅梅', age: 17 };
console.log(person[key]);                     // 韩梅梅 ← 变量作键
person[key] = '李雷';                        // 动态写入
console.log(person.name);                     // 李雷

// 动态调用方法:Reflect.apply 或函数引用
const tools = {
  add(a, b) { return a + b; },
};
const method = 'add';
console.log(tools[method](1, 2));             // 3 ← 方法名也可以是变量
console.log(Reflect.apply(tools.add, tools, [3, 4])); // 7

// 构造反射:Reflect.construct 等价 new
class Point {
  constructor(x, y) { this.x = x; this.y = y; }
}
const p = Reflect.construct(Point, [1, 2]);
console.log(p instanceof Point);              // true

// 原型反射:
console.log(Reflect.getPrototypeOf(p) === Point.prototype); // true
```

§11.6 动态特性:in、delete、原型操纵与遍历深度

```
// in 与 hasOwnProperty 的分工
const proto = { inherited: true };
const own = Object.create(proto);
own.self = 1;
console.log('self' in own);           // true(自己的)
console.log('inherited' in own);      // true(in 沿原型链)
console.log(own.hasOwnProperty('inherited')); // false(不是自己的)
console.log(Object.hasOwn(own, 'inherited')); // false ← ES2022 推荐写法

// delete:删除自身属性
delete own.self;
console.log('self' in own);           // false
// delete 不作用于原型属性:own 仍能看到 inherited

// 通用深度遍历:反射 + 递归(生产日志/调试/测试断言的基础)
function deepDump(obj, prefix = '') {
  if (obj === null || typeof obj !== 'object') {
    console.log(prefix + ' = ' + String(obj));
    return;
  }
  for (const [k, v] of Object.entries(obj)) {
    deepDump(v, prefix + '.' + k);
  }
}
deepDump({ a: { b: 1 }, c: [2, 3] });
// .a.b = 1
// .c.0 = 2
// .c.1 = 3

// 循环引用保护:遍历时记录已访问对象
function safeDump(obj, seen = new Set()) {
  if (obj === null || typeof obj !== 'object') return String(obj);
  if (seen.has(obj)) return '[循环]';
  seen.add(obj);
  const out = {};
  for (const [k, v] of Object.entries(obj)) {
    out[k] = safeDump(v, seen);
  }
  return JSON.stringify(out);
}
const cyc = { name: '环' };
cyc.self = cyc;
console.log(safeDump(cyc)); // {"name":"环","self":"[循环]"}

```

§11.7 生产判断:何时用反射,何时不用

场景	推荐	理由
序列化(对象→JSON/存储)	反射(entries/递归)	通用、字段增删免维护
表单校验(动态字段)	反射 + schema 校验	字段集合运行时才知道
ORM/数据层映射	反射 + 元数据(第12章)	框架层必须通用
调试/日志/测试断言	反射遍历打印	对象结构不确定
业务热路径(每秒万次)	静态访问 obj.field	反射慢 1-2 个数量级
固定结构处理	显式代码,不用反射	可读性、类型安全、编辑器补全
对象内部逻辑	this.xxx 直接访问	反射只用于"通用代码"

```
// 热路径对比(简单量化):
function bench() {
  const obj = { a: 1 };
  const start = Date.now();
  for (let i = 0; i < 1e6; i++) obj.a++;           // 静态访问
  const mid = Date.now();
  for (let i = 0; i < 1e6; i++) obj['a']++;        // 动态访问(方括号)
  const end = Date.now();
  console.log(`静态:${mid - start}ms 动态:${end - mid}ms`);
}
// bench(); // 动态通常慢 1.5-3 倍;带字符串拼接/Reflect 更慢

// 替代手段清单(不用反射也能达到目的):
// ① 固定结构 → 显式字段访问 + TypeScript 类型
// ② 需要通用能力 → 生成代码/模板(第12章 go:generate 哲学)
// ③ 需要动态校验 → 用校验库(zod/joi),内部仍是反射但经过优化
console.log('判断口诀:通用性优先用反射,确定性优先用静态');
```

⚠ 常见误区 11.3:用反射遍历"大对象"做热路径序列化

生产事故:把反射序列化器用于每请求一次的响应体(对象几千字段),QPS 直接腰斩。缓解:① 缓存"键列表"(对象形状不变时,字段集合不变, `Object.keys` 结果可缓存);② 手写热点对象的结构化序列化;③ 用 Fastify 的 schema 序列化(编译成代码)。反射可以缓存,不能盲目信任每次调用。

本章速查表

主题	速查要点
typeof 结果	number/bigint/string/boolean/undefined/symbol/object/function 八种
typeof null	'object'(bug);判 null 用 === null
typeof 数组	'object';判数组用 Array.isArray(跨 realm 安全)
instanceof	沿原型链查;Ctor.prototype 在链上即 true;原始值恒 false
Object.keys	自身可枚举字符串键(不含继承/Symbol/不可枚举)
Object.entries	[k,v] 数组;配 for...of 是最常用遍历;fromEntries 可还原
Object.assign	浅拷贝合并;现代用 {...a, ...b}
Reflect.ownKeys	最全键(含 Symbol 与不可枚举)
描述符三开关	writable 可改 / enumerable 可见 / configurable 可删可重定义
defineProperty	生产:敏感字段 enumerable:false 防泄露;配置 writable:false 防改写
Object.hasOwn	ES2022,替代 hasOwnProperty 的安全写法
in 运算符	沿原型链查;hasOwn 只查自身
delete	删自身属性;不删原型属性;configurable:false 时无效
深度遍历	递归 + entries + Set 防循环引用
性能	动态访问比静态慢 1.5–3 倍;热路径别用反射,或缓存键列表
生产判断	框架/序列化/校验用反射;业务固定结构用静态

最小必会清单

- 能写出 typeof 八种结果,并说出 null/数组/函数三个特例
- 能说出 instanceof 的原理(原型链)与跨 realm 陷阱
- 能用 keys/values/entries + for...of 遍历对象
- 能说出 keys 的三种"看不见"(继承/Symbol/不可枚举)及对应 API
- 能用 defineProperty 创建只读 + 不可枚举字段
- 能说出 Object.freeze 与 defineProperty 的关系
- 能用 Reflect.ownKeys 拿到最全键集合
- 能写出带循环引用保护的深度遍历
- 能说出反射慢于静态访问的量级与缓存手段
- 能按"通用性 vs 确定性"判断何时用反射

自测题

Q 1. 写出输出:console.log(typeof null, typeof [], typeof () => {}), Array.isArray([]))

✓ 参考答案

'object'、'object'、'function'、true。null 是 bug;数组也是 object;箭头函数是 function。

Q 2. `class A {};` `const a = new A();` 请写出 `a instanceof A`、`a instanceof Object` 的结果并解释。

✓ 参考答案

true、true。instanceof 沿原型链查找:A.prototype 在 a 的链上,Object.prototype 也在(链的终点),所以都匹配。

Q 3. `Object.keys(obj)` 看不到哪三类属性?分别用什么 API?

✓ 参考答案

继承属性(for...in 或沿原型遍历)、Symbol 键(Reflect.ownKeys/getOwnPropertySymbols)、不可枚举属性(getOwnPropertyNames/Reflect.ownKeys)。

Q 4. 如何让一个字段"代码里能读,但 `JSON.stringify` 和日志里不出现"?

✓ 参考答案

defineProperty 设置 enumerable: false,JSON.stringify 与 Object.keys 只序列化可枚举属性。

Q 5. `delete obj.x` 返回什么?在什么情况下返回 false?

✓ 参考答案

成功删除返回 true;configurable: false 的属性、不可删除的(如非严格模式的某些内建)返回 false。

Q 6. 遍历一个可能循环引用的对象树,如何防止死循环?

✓ 参考答案

用 Set 记录已访问对象,递归前检查,命中则标记 '[循环]' 并返回。

章末实验题:对象内省工具箱(通用序列化器)

实验 11:对象内省工具箱(序列化器 + 遍历打印机)

实验目标:实现通用"对象属性遍历打印机"与手写 JSON 序列化器,覆盖全章反射 API。

实验要求:

- 任务 1(覆盖:Object.entries、递归):dumpObject(obj) 深度打印所有可枚举字段,缩进表示层级
- 任务 2(覆盖:循环引用防护、Set):遍历带自引用的对象不死循环,输出 [循环] 标记
- 任务 3(覆盖:typeof、Array.isArray、typeof 工具):serialize(obj) 手写 JSON:支持 null/布尔/数字/字符串/数组/对象,递归序列化
- 任务 4(覆盖:字符串转义、不可枚举):字符串值转义引号与换行;用 defineProperty 制造不可枚举字段,序列化器必须忽略它(与 JSON.stringify 行为一致)
- 任务 5(覆盖:Reflect.ownKeys、描述符):inspectProperty(obj, key) 输出该属性完整描述符(三开关)
- 任务 6(覆盖:getOwnPropertyNames、跨原型):diffKeys(a, b) 用 getOwnPropertyNames 对比两对象自身键差异
- 任务 7(覆盖:Object.fromEntries、动态属性):把序列化结果解析回来(用 fromEntries 重建),验证往返一致性
- 任务 8(覆盖:生产判断):注释说明序列化器与 JSON.stringify 的行为差异,以及何时该用哪个

实验环境:Node.js 20+,运行 `node introspect.js`。

第一步 **写类型工具:**typeof 区分 null/array/object,序列化主函数按类型分发

第二步 **写递归:**对象序列化 = 遍历 entries → 递归每个值 → 拼 JSON 片段;数组同理

第三步 **后做演示:**构造含嵌套/循环/不可枚举/特殊字符的对象,验证与 JSON.stringify 输出对比

✅ 参考答案要点

核心:typeof 分发 + 递归序列化 + 转义 + 防循环 + 描述符内省。约 130 行,覆盖全章反射 API。

```
// introspect.js — Node.js 20+ 运行:node introspect.js
'use strict';

// 1. 类型工具(覆盖:typeof、Array.isArray、null 特判)
function typeOf(x) {
  if (x === null) return 'null';
  if (Array.isArray(x)) return 'array';
  const t = typeof x;
  if (t === 'object') return 'object';
  if (t === 'function') return 'function';
  return t;
}

// 2. 深度遍历打印(覆盖:Object.entries、递归、缩进)
function dumpObject(obj, depth = 0, seen = new Set()) {
  const indent = ' '.repeat(depth);
  if (obj === null || typeof obj !== 'object') {
    console.log(indent + String(obj));
    return;
  }
  if (seen.has(obj)) {
    console.log(indent + '[循环]');
    return;
  }
  seen.add(obj);
  for (const [k, v] of Object.entries(obj)) { // 只遍历可枚举自身属性
    console.log(indent + k + ':');
    dumpObject(v, depth + 1, seen);
  }
}

// 3. 手写 JSON 序列化器(覆盖:typeOf 分发、递归、转义)
function serialize(value) {
  switch (typeOf(value)) {
    case 'null':
      return 'null';
    case 'number':
      if (!Number.isFinite(value)) return 'null'; // NaN/Infinity → null(与内置一致)
      return String(value);
    case 'boolean':
      return String(value);
    case 'string':
      return '"' + escapeString(value) + '"';
    case 'array': {
      const parts = value.map((v) => serialize(v)); // undefined 元素 → null(简化)
      return '[' + parts.join(',') + ']';
    }
    case 'object': {
      const parts = [];
      for (const [k, v] of Object.entries(value)) {
        if (v === undefined) continue; // undefined 属性丢弃(与内置一致)
        parts.push('"' + escapeString(k) + '":' + serialize(v));
      }
      return '{' + parts.join(',') + '}';
    }
  }
}
```

```

    default:
        return 'null'; // function/symbol → null(丢弃语义)
    }
}
// 字符串转义(覆盖:反斜杠、引号、换行、制表符)
function escapeString(s) {
    return s
        .replace(/\\/g, '\\\\')
        .replace(/"/g, '\\"')
        .replace(/\n/g, '\\n')
        .replace(/\t/g, '\\t')
        .replace(/\r/g, '\\r');
}

// 4. 属性描述符内省(覆盖:getOwnPropertyDescriptor、Reflect.ownKeys)
function inspectProperty(obj, key) {
    const desc = Object.getOwnPropertyDescriptor(obj, key);
    if (!desc) return `属性 ${key} 不存在`;
    return {
        key,
        writable: desc.writable,
        enumerable: desc.enumerable,
        configurable: desc.configurable,
    };
}

// 5. 键集合对比(覆盖:getOwnPropertyNames、跨原型)
function diffKeys(a, b) {
    const ka = Object.getOwnPropertyNames(a);
    const kb = new Set(Object.getOwnPropertyNames(b));
    const onlyInA = ka.filter((k) => !kb.has(k));
    return onlyInA;
}

// 6. 演示对象(覆盖:嵌套、循环、不可枚举、特殊字符)
function buildSample() {
    const sample = {
        name: '李雷 "引号"',
        note: '换行\n与制表\t符',
        score: 92.5,
        ok: true,
        tags: ['a', 'b', null],
        nested: { deep: { value: 1 } },
    };
    Object.defineProperty(sample, 'secret', {
        value: '隐藏字段', enumerable: false, writable: false,
    });
    sample.self = sample; // 循环引用
    return sample;
}

// 7. 主流程
function main() {
    const sample = buildSample();

```

```

console.log('== 深度遍历(循环引用防护) ==');
dumpObject(sample);

console.log('== 手写序列化 vs 内置 JSON.stringify ==');
const mine = serialize(sample);
console.log('mine:', mine.slice(0, 120) + '...(截断)');
console.log('内置:', JSON.stringify(sample).slice(0, 120) + '...(截断)');

const clean = { name: '韩梅梅', scores: [1, 2, 3], ok: true };
console.log('往返一致性:', serialize(clean) === JSON.stringify(clean));

console.log('== 属性描述符自省 ==');
console.log(inspectProperty(sample, 'secret')); // writable:false 等
console.log(inspectProperty(sample, 'name'));

console.log('== 键对比 ==');
const other = { name: 'x', extra: 1 };
console.log('sample 独有键:', diffKeys(sample, other).join(', '));
}

main();

```

设计说明:serialize 与内置 JSON.stringify 的行为对齐点:NaN/Infinity→null、undefined 属性丢弃、不可枚举忽略、函数/符号→null;差异点:不支持 toJSON 钩子与缩进参数——注释里注明"生产用 JSON.stringify,手写版用于理解反射"。dumpObject 用 Set 防循环;inspectProperty 输出三开关,呼应 §11.4;diffKeys 演示 getOwnPropertyNames 的全量语义。运行后对比"mine"与"内置"两个输出,可直观看到手写序列化器与标准行为的一致与差异。

下一章预告:第12章 注解与装饰器 —— 高阶函数式装饰、TC39 装饰器提案现状与 TypeScript 装饰器,以及计时/重试/缓存的完整实现。